



Chitkara

PAPER OF CSE-2025 - PYTHON TEST11 EXAM

Q1. Concrete Batch Mixer Cycles

A construction site uses a **concrete mixer** that prepares batches for various sections of a project. The mixer can produce **1000 units** of concrete per full mixing cycle. The total cycles required are tracked. If the concrete requirement is not a multiple of 1000 units, an **extra cycle** must be performed for the final, partial batch.

You must: Calculate the **total number of mixing cycles** needed for a list of required concrete quantities.

Input Format:

The input is a list of integers Q (concrete quantities in units).

Output Format:

Print the total cycles values.

Sample Input:

```
[1000, 1001, 1999, 2000, 800]
```

Sample Output:

```
8
```

Explanation:

For each quantity, cycles needed are:

1000 units → 1 cycle

1001 units → 2 cycles

1999 units → 2 cycles

2000 units → 2 cycles

800 units → 1 cycle

Total = 1 + 2 + 2 + 2 + 1 = 8

ANSWER:

Test case 1 :

Input:

[1000, 1001, 1999, 2000, 800]

Output:

8

Test case 2 :

Input:

[1000, 2000, 3000]

Output:

6

Test case 3 :

Input:

[999, 1, 1001]

Output:

4

Test case 4 :

Input:

[5000]

Output:

5

Test case 5 :

Input:

[]

Output:

0

Test case 6 :

Input:

[0, 200, 400]

Output:

2

Q2. Chemical Reagent Vials Count

A lab technician is preparing a set of experiments, each requiring a specific amount of **reagent**. The reagent is stored in **vials**, and each vial contains exactly **1000 milliliters (ml)**. To cover the required amount, the technician always uses the minimum number of full or partial vials. If the required amount is not perfectly divisible by 1000 ml, an **additional vial** must be opened for the remainder.

You must: Compute & print the **total number of vials** needed for a list of required reagent amounts.

Input Format:

A list of integers 'm', representing the required reagent amounts (in ml).

Output Format:

Output the total number of vials needed for a list.

Sample Input:

[1500, 1999, 2900, 2000]

Sample Output:

9

Explanation:

1500 ml $\rightarrow$ 2 vials

1999 ml $\rightarrow$ 2 vials

2900 ml $\rightarrow$ 3 vials

2000 ml $\rightarrow$ 2 vials

Total = 2 + 2 + 3 + 2 = 9

ANSWER:

Test case 1 :

Input:

[1500, 1999, 2900, 2000]

Output:

9

Test case 2 :

Input:

[1000, 2000, 3000]

Output:

6

Test case 3 :

Input:

[999, 1, 1001]

Output:

4

Test case 4 :

Input:

[5000]

Output:

5

Test case 5 :

Input:

[]

Output:

0

Test case 6 :

Input:

[0, 200, 400]

Output:

2

Q3. Network Data Packet Transmission

A network device transmits large files in fixed-size **data packets**. Each packet holds a maximum of **1000 bytes** of data. For a file of any size, the minimum number of packets is prepared. If the file size is not a multiple of 1000 bytes, an **extra packet** is created for the remaining data.

You have to determine the **total number of packets** required to transmit a list of files, given their sizes in bytes.

Input Format:

A list of integers B, representing file sizes (in bytes).

Output Format:

Output the total number of production runs needed

Sample Input:

[4000, 1, 1000]

Sample Output:

6

Explanation:

For each file size, packets needed are:

4000 bytes $\rightarrow$ 4 packets

1 bytes $\rightarrow$ 1 packets

1000 bytes $\rightarrow$ 1 packets

Total required shelves: $4 + 1 + 1 = 6$ packets

ANSWER:

Test case 1 :

Input:

[4000, 1, 1000]

Output:

6

Test case 2 :

Input:

[1000, 2000, 3000]

Output:

6

Test case 3 :

Input:

[999, 1, 1001]

Output:

4

Test case 4 :

Input:

[5000]

Output:

5

Test case 5 :

Input:

[]

Output:

0

Test case 6 :

Input:

[0, 200, 400]

Output:

2

Q4. Patient Visit Counter

A hospital tracks patient visits to different specialists across various departments. Records map **Patient ID** to the **Number of Visits** in a week. Manual transcription errors lead to some visit counts being recorded as text.

You must:

Compute the **Total Visits** for each **Patient ID**.

Invalid visit counts (non-integers) must be skipped with output message **"Invalid visit counts"**.

Output the final aggregated visit counts.

Input Format:

A list of dictionaries: [{ Patient ID : Visit Count},]

Output Format:

A single dictionary with values : { Patient ID : Total Visit Count,}

Sample Input 1:

```
[{'P1': 1}, {'P2': 3, 'P1': 2}]
```

Sample Output 1:

```
{'P1': 3, 'P2': 3}
```

Sample Input 2:

```
[{'P3': 5, 'P4': 1}, {'P3': 'one', 'P5': 2}]
```

Sample Output 2:

```
Invalid visit counts
{'P3': 5, 'P4': 1, 'P5': 2}
```

ANSWER:

Test case 1 :

Input:

```
[{'P1': 1}, {'P2': 3, 'P1': 2}]
```

Output:

```
{'P1': 3, 'P2': 3}
```

Test case 2 :

Input:

```
[{'A': 100}, {'A': 200}]
```

Output:

```
{'A': 300}
```

Test case 3 :

Input:

```
[{'A': 100}, {'B': 'x'}]
```

Output:

Invalid visit counts

```
{'A': 100}
```

Test case 4 :

Input:

```
[{'A': 0}, {'B': 0}]
```

Output:

```
{'A': 0, 'B': 0}
```

Test case 5 :

Input:

`[{'A':50},{'A':50},{'A':50}]`

Output:

`{'A': 150}`Test case 6 :

Input:

`[{'A':10,'B':20},{'A':'10'}]`

Output:

Invalid visit counts

`{'A': 10, 'B': 20}`**Q5. Venture Capital Investment**

A Venture Capital firm logs investments into startups. Entries map **Startup Name** to the **Investment Amount** (in millions). Due to fractional funding and foreign currencies, some amounts are logged as text.

You must:

Calculate the Total Investment Amount for each Startup Name.

If an investment amount is **not an integer**, skip that record and output **"Investment amount invalid"**.

Display the aggregated investment totals.

Input Format:

A list of dictionaries: [{ Startup : Amount },]

Output Format:

A single dictionary with values : { Startup: Total Investment,}

Sample Input 1:

```
[{'TechX': 50}, {'BioY': 20, 'TechX': 10}]
```

Sample Output 1:

```
{'TechX': 60, 'BioY': 20}
```

Sample Input 2:

```
[{'CodeQuotient': 100, 'CodingNinza': 30}, {'FinZ': 'hundred', 'AutoV': 50}]
```

Sample Output 2:

```
Investment amount invalid  
{'CodeQuotient': 100, 'CodingNinza': 30, 'AutoV': 50}
```

ANSWER:

Test case 1 :

Input:

```
[{'TechX': 50}, {'BioY': 20, 'TechX': 10}]
```

Output:

```
{'TechX': 60, 'BioY': 20}
```

Test case 2 :

Input:

```
[{'A':100},{ 'A':200}]
```

Output:

```
{ 'A': 300 }
```

Test case 3 :

Input:

```
[{'A':100},{'B':'x'}]
```

Output:

Investment amount invalid

```
{'A': 100}
```

Test case 4 :

Input:

```
[{'A':0},{'B':0}]
```

Output:

```
{'A': 0, 'B': 0}
```

Test case 5 :

Input:

```
[{'A':50},{'A':50},{'A':50}]
```

Output:

```
{'A': 150}
```

Test case 6 :

Input:

```
[{'A':10,'B':20},{'A':'10'}]
```

Output:

Investment amount invalid

```
{'A': 10, 'B': 20}
```

Q6. Football Match Goals Scorer

A football league tracks goals scored during a tournament, recorded after each match. The logs map **Player ID** to the **Goals Scored** in that match. Occasionally, errors occur where the goal count is noted incorrectly.

You must:

Determine the Total Goals Scored by each Player ID.

If a goal count is not an integer, skip that record and print **"Goal count error, ignoring match data"** message.

Display the final player goal totals.

Input Format:

A list of dictionaries: [{ Player ID : Goals },]

Output Format:

A single dictionary with values : { Player ID: Total Goals,}

Sample Input 1:

```
[{'Ronaldo': 2}, {'Messi': 1, 'Ronaldo': 3}]
```

Sample Output 1:

```
{'Ronaldo': 5, 'Messi': 1}
```

Sample Input 2:

```
[{'Neymar': 1, 'Mbappe': 2}, {'Neymar': 'one', 'Salah': 3}]
```

Sample Output 2:

```
Goal count error, ignoring match data
{'Neymar': 1, 'Mbappe': 2, 'Salah': 3}
```

ANSWER:Test case 1 :

Input:

```
[{'Ronaldo': 2}, {'Messi': 1, 'Ronaldo': 3}]
```

Output:

```
{'Ronaldo': 5, 'Messi': 1}
```

Test case 2 :

Input:

```
[{'A':100},{'A':200}]
```

Output:

```
{'A': 300}
```

Test case 3 :

Input:

```
[{'A':100},{'B':'x'}]
```

Output:

```
Goal count error, ignoring match data
{'A': 100}
```

Test case 4 :

Input:

```
[{'A':0},{'B':0}]
```

Output:

```
{'A': 0, 'B': 0}
```

Test case 5 :

Input:

```
[{'A':50},{'A':50},{'A':50}]
```

Output:

```
{'A': 150}
```

Test case 6 :

Input:

```
[{'A':10,'B':20},{'A':'10'}]
```

Output:

Goal count error, ignoring match data

{'A': 10, 'B': 20}

Q7. Reading Speed Assessment

A teacher assesses student reading speeds by measuring the time taken (in seconds) to read a standard passage.

A time of 45 seconds or less is rated "Excellent". A time greater than 45 seconds is rated "Standard". The test batch is marked by a reading of -1 (end of class).

Your Task:

Given a list of reading times, count the number of "Excellent" readers (≤ 45) and "Standard" readers (> 45). Stop processing at the batch marker (-1).

Input Format:

List of integers representing reading times (ends with -1).

Output Format:

Tuple (standard_count, excellent_count)

Note: You have to return the result tuple.

Sample Test Case 1:

Input:

[55, 35, 20, 99, 45, -1]

Output:

(2, 3)

Sample Test Case 2:

Input:

[19, 23, 30, 35, -1]

Output:

(0, 4)

ANSWER:

Test case 1 :

Input:

[55, 35, 20, 99, 45, -1]

Output:

(2, 3)

Test case 2 :

Input:

[10, 15, 25, -1]

Output:

(0, 3)

Test case 3 :

Input:

[35, 40, 50, -1]

Output:

(1, 2)

Test case 4 :

Input:

[30, 29, 31, -1]

Output:

(0, 3)

Test case 5 :

Input:

[5, 10, 15, 20, 25, -1]

Output:

(0, 5)

Test case 6 :

Input:

[55, 60, 10, 5, -1]

Output:

(2, 2)

Q8. Game Score Streak Analyze

An arcade game tracks player scores. A "Low Score" streak is maintained if the player scores **30 points or less** on a single level (indicating bad play). Any score over 30 is considered "Normal". The game log terminates with a score of **-1** (game over).

Task:

Process the list of level scores.

- Count the number of "Low Score" levels (≤ 30) and "Normal" levels (> 30).
- Stop when the game over marker is found.

Input Format:

List of integers representing component lengths (ends with -1).

Output Format:

Tuple (low_count, normal_count)

Note: You have to return the result tuple.

Sample Test Case 1:**Input:**

[40, 50, 60, -1]

Output:

(0, 3)

Sample Test Case 2:**Input:**

[14, 39, 30, 48, 21, -1]

Output:

(3, 2)

ANSWER:Test case 1 :

Input:

[40, 50, 60, -1]

Output:

(0, 3)

Test case 2 :

Input:

[10, 15, 25, -1]

Output:

(3, 0)

Test case 3 :

Input:

[35, 40, 50, -1]

Output:

(0, 3)

Test case 4 :

Input:

[30, 29, 31, -1]

Output:

(2, 1)

Test case 5 :

Input:

[5, 10, 15, 20, 25, -1]

Output:

(5, 0)

Test case 6 :

Input:

[45, 60, 10, 5, -1]

Output:

(2, 2)

Q9. Temperature Fluctuation Analysis

A research station monitors a chemical reactor's temperature every minute. The **safe operating temperature** is defined as 30°C or lower. Any reading above 30°C is considered a "Critical" fluctuation. The sensor stops logging data when it registers a value of **-1** (sensor failure).

Task:

Write a function that accepts a list of temperature readings (integers). Classify each reading:

- If temperature ≤ 30 then "Normal"
- If temperature > 30 then "Critical"
- Stop processing at -1 and return the final counts of Normal and Critical readings as tuple.

Input Format:

List of integers representing the temperature readings (ends with -1).

Output Format:

Tuple (normal_count, critical_count)

Sample Test Case 1:

Input:

[25, 30, 31, 28, 40, -1]

Output:

(3, 2)

Sample Test Case 2:

Input:

```
[35, 45, -1]
```

Output:

```
(0, 2)
```

ANSWER:Test case 1 :

Input:

```
[25, 30, 31, 28, 40, -1]
```

Output:

```
(3, 2)
```

Test case 2 :

Input:

```
[10, 15, 25, -1]
```

Output:

```
(3, 0)
```

Test case 3 :

Input:

```
[35, 40, 50, -1]
```

Output:

```
(0, 3)
```

Test case 4 :

Input:

```
[30, 29, 31, -1]
```

Output:

```
(2, 1)
```

Test case 5 :

Input:

```
[5, 10, 15, 20, 25, -1]
```

Output:

```
(5, 0)
```

Test case 6 :

Input:

```
[45, 60, 10, 5, -1]
```

Output:

```
(2, 2)
```